Pontificia Universidad Católica Argentina

Modelos y Simulación

Manual de Prácticas · Ingeniería en Informática

Alumno / equipo		**Fecha**	5/11/2026, 12:04:14 a.m.
Práctica	14 · Modelo híbrido		

Objetivo

Integrar predictor, reglas, Monte Carlo y simulación de eventos en una arquitectura modular.

Situación problemática

La predicción de demanda alimenta el simulador y una regla decide capacidad.

Código desarrollado

```
# Práctica 14 · Modelo híbrido
# El predictor y las utilidades de simulación ya están disponibles.

SEMILLA = 2026
UMBRAL_1_SERVIDOR = 700
UMBRAL_2_SERVIDORES = 850
ERROR_MAX = 0.15
REPLICAS = 200
UTILIZACION_MAX = 0.6

def decidir_capacidad(demanda):
    if demanda < UMBRAL_1_SERVIDOR:
        return 1
    elif demanda < UMBRAL_2_SERVIDORES:
        return 2
    return 4

def servidores_minimos(datos):
    trabajo = 0
    for d in datos:
        trabajo = trabajo + d["servicio"]
    carga = trabajo / datos[len(datos) - 1]["llegada"]
    return int(carga / UTILIZACION_MAX) + 1

horas = [6, 9, 12, 15, 18, 21]

for hora in horas:
    demanda = int(predecir_demanda(hora))

    # TODO 1: decidí la cantidad de servidores según la demanda.
    servidores = decidir_capacidad(demanda)

    # TODO 2: generá los logs del escenario con semilla reproducible.
    datos = generar_logs(demanda, SEMILLA + hora)

    # TODO 3: simulá y obtené el P95.
    p95 = round(simular_cola(datos, servidores)["p95"], 2)

    print("Hora:", hora, "Demanda:", demanda, "Servidores:", servidores, "P95:", p95)

# TODO 4: explicá cómo se propaga el error del predictor hacia la decisión de capacidad.
# El error del predictor cambia la demanda que ve la regla. Como la regla es un escalón
# (700 y 850), un error chico solo importa si cruza un umbral: ahí cambia la cantidad de
# servidores y, con ella, el P95. Lejos de los umbrales el error no cambia la decisión.
print("---")
```

```
print("Contratos de datos (verificación):")
contrato_demanda = True
contrato_regla = True
contrato_logs = True
contrato_resultado = True
for hora in horas:
    demanda = predecir_demanda(hora)
    if demanda <= 0:
        contrato_demanda = False
    if decidir_capacidad(int(demanda)) != decidir_servidores(int(demanda)):
        contrato_regla = False
    datos = generar_logs(int(demanda), SEMILLA + hora)
    if len(datos) != int(demanda):
        contrato_logs = False
    anterior = 0
    for d in datos:
        if d.get("id") == None or d.get("servicio") == None or d.get("prioridad") == None:
            contrato_logs = False
        if d["llegada"] < anterior or d["servicio"] <= 0:
            contrato_logs = False
        anterior = d["llegada"]
    r = simular_cola(datos, 2)
    if r["p95"] < 0 or len(r["latencias"]) != len(datos):
        contrato_resultado = False
print("Predictor devuelve demanda positiva:", contrato_demanda)
print("Regla propia coincide con decidir_servidores():", contrato_regla)
print("Logs con campos completos, llegadas ordenadas y cantidad igual a la demanda:", contrato_logs)
print("Simulador devuelve una latencia por solicitud:", contrato_resultado)

print("---")
print("Propagación del error del predictor: error relativo triangular de -15 % a +15 %,", REPLICAS, "réplicas por hora")
rng_seed(SEMILLA)
for hora in horas:
    demanda = int(predecir_demanda(hora))
    serv_plan = decidir_capacidad(demanda)
    cambios = 0
    suma_plan = 0
    suma_real = 0
    for rep in range(REPLICAS):
        error = triangular(-ERROR_MAX, ERROR_MAX, 0)
        demanda_real = int(demanda * (1 + error))
        serv_real = decidir_capacidad(demanda_real)
        datos = generar_logs(demanda_real, 1000 + rep)
        suma_plan = suma_plan + simular_cola(datos, serv_plan)["p95"]
        suma_real = suma_real + simular_cola(datos, serv_real)["p95"]
        if serv_real != serv_plan:
            cambios = cambios + 1
    print("Hora:", hora, "Demanda prevista:", demanda, "Servidores:", serv_plan,
```

```
          "| decisión distinta a la de la demanda real:", round(cambios / REPLICAS * 100, 1), "%",
          "| P95 medio con la decisión del predictor:", round(suma_plan / REPLICAS, 1),
          "| con la decisión de la demanda real:", round(suma_real / REPLICAS, 1))

print("---")
print("Comparación con el modelo base (2 servidores fijos todo el día):")
total_hibrido = 0
total_base = 0
for hora in horas:
    demanda = int(predecir_demanda(hora))
    serv_h = decidir_capacidad(demanda)
    datos = generar_logs(demanda, SEMILLA + hora)
    p95_h = round(simular_cola(datos, serv_h)["p95"], 2)
    p95_b = round(simular_cola(datos, 2)["p95"], 2)
    total_hibrido = total_hibrido + serv_h
    total_base = total_base + 2
    print("Hora:", hora, "| híbrido:", serv_h, "servidores, P95", p95_h, "| base: 2 servidores, P95", p95_b)
print("Servidores-hora (suma de servidores en las 6 franjas): híbrido", total_hibrido, "| base", total_base)

# DESAFÍO: cambio estructural en la regla de capacidad.
# La regla deja de depender solo de la demanda: se agrega un mínimo de servidores
# calculado desde los propios logs para que la utilización no supere 0.6.
# Por eso el modelo cambia: ahora hay que generar los logs ANTES de decidir.
print("---")
print("DESAFÍO: regla con mínimo de estabilidad calculado desde los logs")
total_nueva = 0
for hora in horas:
    demanda = int(predecir_demanda(hora))
    datos = generar_logs(demanda, SEMILLA + hora)
    minimo = servidores_minimos(datos)
    serv_n = max([decidir_capacidad(demanda), minimo])
    p95_n = round(simular_cola(datos, serv_n)["p95"], 2)
    total_nueva = total_nueva + serv_n
    print("Hora:", hora, "Demanda:", demanda, "Mínimo por logs:", minimo, "Servidores:", serv_n, "P95:", p95_n)
print("Servidores-hora: regla nueva", total_nueva, "| híbrido original", total_hibrido, "| base", total_base)
```

Salida / evidencia

```
Hora: 6 Demanda: 600 Servidores: 1 P95: 600.56
Hora: 9 Demanda: 677 Servidores: 1 P95: 831.16
Hora: 12 Demanda: 859 Servidores: 4 P95: 7.41
Hora: 15 Demanda: 889 Servidores: 4 P95: 7.71
Hora: 18 Demanda: 750 Servidores: 2 P95: 11.92
Hora: 21 Demanda: 600 Servidores: 1 P95: 693.22
---
Contratos de datos (verificación):
Predictor devuelve demanda positiva: True
Regla propia coincide con decidir_servidores(): True
Logs con campos completos, llegadas ordenadas y cantidad igual a la demanda: True
Simulador devuelve una latencia por solicitud: True
---
Propagación del error del predictor: error relativo triangular de -15 % a +15 %, 200 réplicas por hora
Hora: 6 Demanda prevista: 600 Servidores: 1 | decisión distinta a la de la demanda real: 0 % | P95 medio con la decisión del predictor: 659.4 | con la decisión de la
demanda real: 659.4
Hora: 9 Demanda prevista: 677 Servidores: 1 | decisión distinta a la de la demanda real: 29 % | P95 medio con la decisión del predictor: 743.6 | con la decisión de la
demanda real: 514.5
Hora: 12 Demanda prevista: 859 Servidores: 4 | decisión distinta a la de la demanda real: 45.5 % | P95 medio con la decisión del predictor: 7.6 | con la decisión de la
demanda real: 10.4
Hora: 15 Demanda prevista: 889 Servidores: 4 | decisión distinta a la de la demanda real: 27 % | P95 medio con la decisión del predictor: 7.6 | con la decisión de la
demanda real: 9.3
Hora: 18 Demanda prevista: 750 Servidores: 2 | decisión distinta a la de la demanda real: 14 % | P95 medio con la decisión del predictor: 13.9 | con la decisión de la
demanda real: 117
Hora: 21 Demanda prevista: 600 Servidores: 1 | decisión distinta a la de la demanda real: 0 % | P95 medio con la decisión del predictor: 658.1 | con la decisión de la
demanda real: 658.1
---
Comparación con el modelo base (2 servidores fijos todo el día):
Hora: 6 | híbrido: 1 servidores, P95 600.56 | base: 2 servidores, P95 12.07
Hora: 9 | híbrido: 1 servidores, P95 831.16 | base: 2 servidores, P95 15.14
Hora: 12 | híbrido: 4 servidores, P95 7.41 | base: 2 servidores, P95 14.02
Hora: 15 | híbrido: 4 servidores, P95 7.71 | base: 2 servidores, P95 15.3
Hora: 18 | híbrido: 2 servidores, P95 11.92 | base: 2 servidores, P95 11.92
Hora: 21 | híbrido: 1 servidores, P95 693.22 | base: 2 servidores, P95 12.38
Servidores-hora (suma de servidores en las 6 franjas): híbrido 13 | base 12
---
DESAFÍO: regla con mínimo de estabilidad calculado desde los logs
Hora: 6 Demanda: 600 Mínimo por logs: 3 Servidores: 3 P95: 8
Hora: 9 Demanda: 677 Mínimo por logs: 3 Servidores: 3 P95: 8.39
Hora: 12 Demanda: 859 Mínimo por logs: 3 Servidores: 4 P95: 7.41
Hora: 15 Demanda: 889 Mínimo por logs: 3 Servidores: 4 P95: 7.71
Hora: 18 Demanda: 750 Mínimo por logs: 3 Servidores: 3 P95: 7.86
Hora: 21 Demanda: 600 Mínimo por logs: 3 Servidores: 3 P95: 7.98
Servidores-hora: regla nueva 20 | híbrido original 13 | base 12
```

Preguntas de interpretación

1. ¿Qué decisión puede apoyarse con esta práctica?

Cuántos servidores asignar por franja y si se debe cambiar la regla de capacidad actual por una que incluya un mínimo por estabilidad.

2. ¿Qué variable o supuesto condiciona más el resultado?

La cantidad de servidores respecto de la carga: con 1 servidor todo se satura, sin importar el error del predictor. En segundo lugar, la distancia entre la demanda prevista y los umbrales de la regla (700 y 850), que define cuánto se propaga el error.

3. ¿Cómo verificaste que la implementación representa lo definido?

Se comprobaron los contratos de datos (todos True), que la regla propia coincide con decidir_servidores(), que dos corridas dan la misma salida, y que la hora 18 da el mismo P95 (11,92) en el híbrido y en el base porque ambos usan 2 servidores con los mismos logs. 3 servidores dan utilización cercana a 0,5 y P95 cercano a 8.

4. ¿Qué resultado cambió respecto de tu expectativa inicial y por qué?

Esperaba que el modelo híbrido mejorara al base porque ajusta la capacidad a la demanda. Pero la regla deja 1 servidor cuando la demanda es baja y el simulador no reduce la carga por unidad de tiempo, así que esas franjas colapsaron (P95 de 600 a 830). También esperaba que el error del predictor fuera el problema principal; resultó secundario frente a la regla mal calibrada.

5. ¿Hasta dónde pueden generalizarse las conclusiones?

Solo para este generador de datos (llegadas constantes), este predictor, estas 6 franjas y el supuesto de error de +-15 %. Si la demanda real cambiara el ritmo de llegadas, la regla de 1 servidor podría ser razonable y habría que hacer devuelta el análisis.

Conclusiones / observaciones

El error del predictor solo afecta la decisión cuando la demanda prevista queda cerca de un límite de la regla (700 o 850). A las 9 la regla asigna menos servidores de los necesarios, y a las 12 y 15 asigna más de los necesarios. Si se agrega un mínimo de servidores calculado con los datos, el P95 queda por debajo de 10 en todas las horas, usando 20 servidores en total contra 12 del modelo base. Recomiendo esa regla si el objetivo es P95 menor a 10, y nunca dejar 1 solo servidor cuando la demanda es baja.

Desafío adicional

Modificar una condición estructural del problema, no solamente un número, y explicar por qué el nuevo escenario requiere cambios en el modelo. Comparar los resultados con la versión base y registrar una recomendación condicionada.

```
Detallado en el informe
```